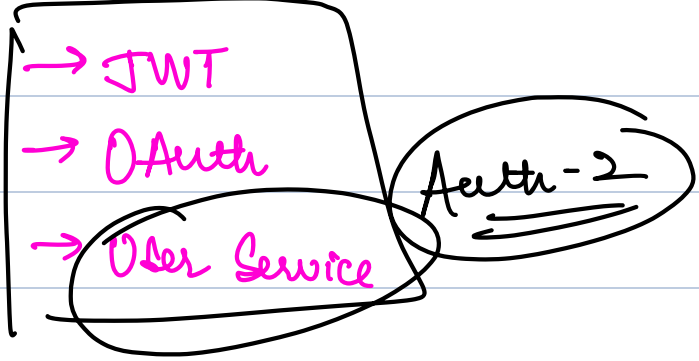
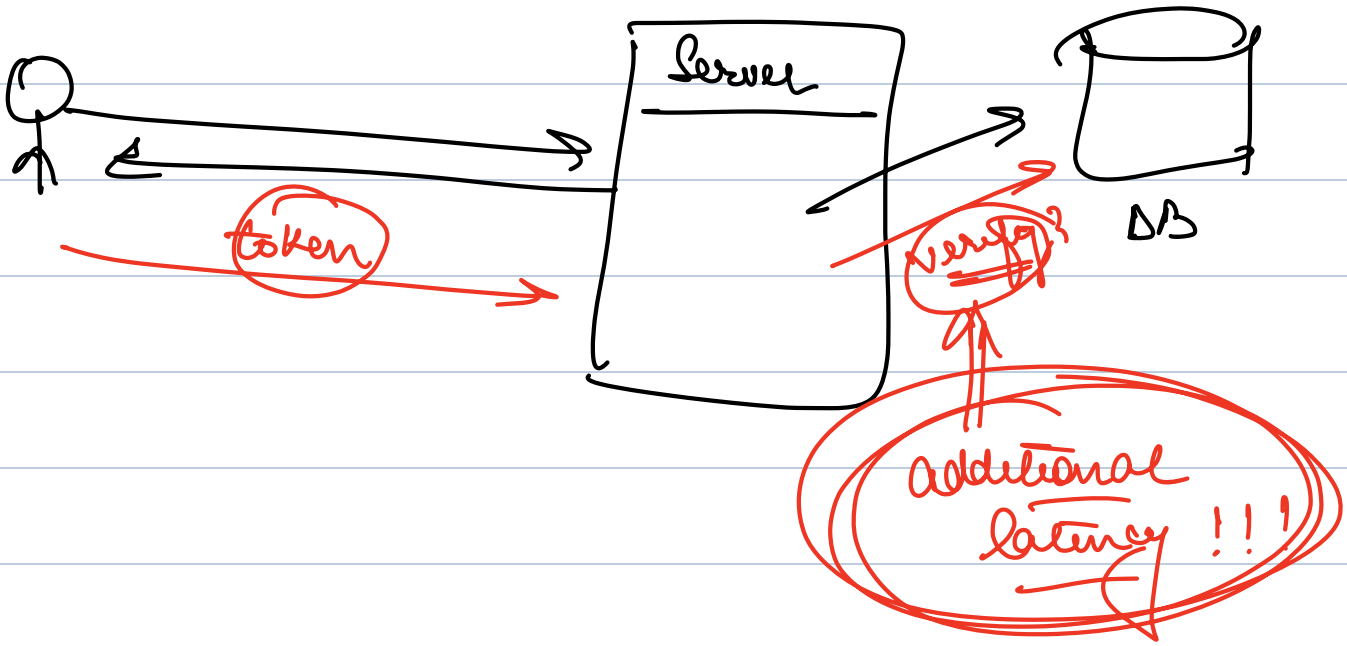
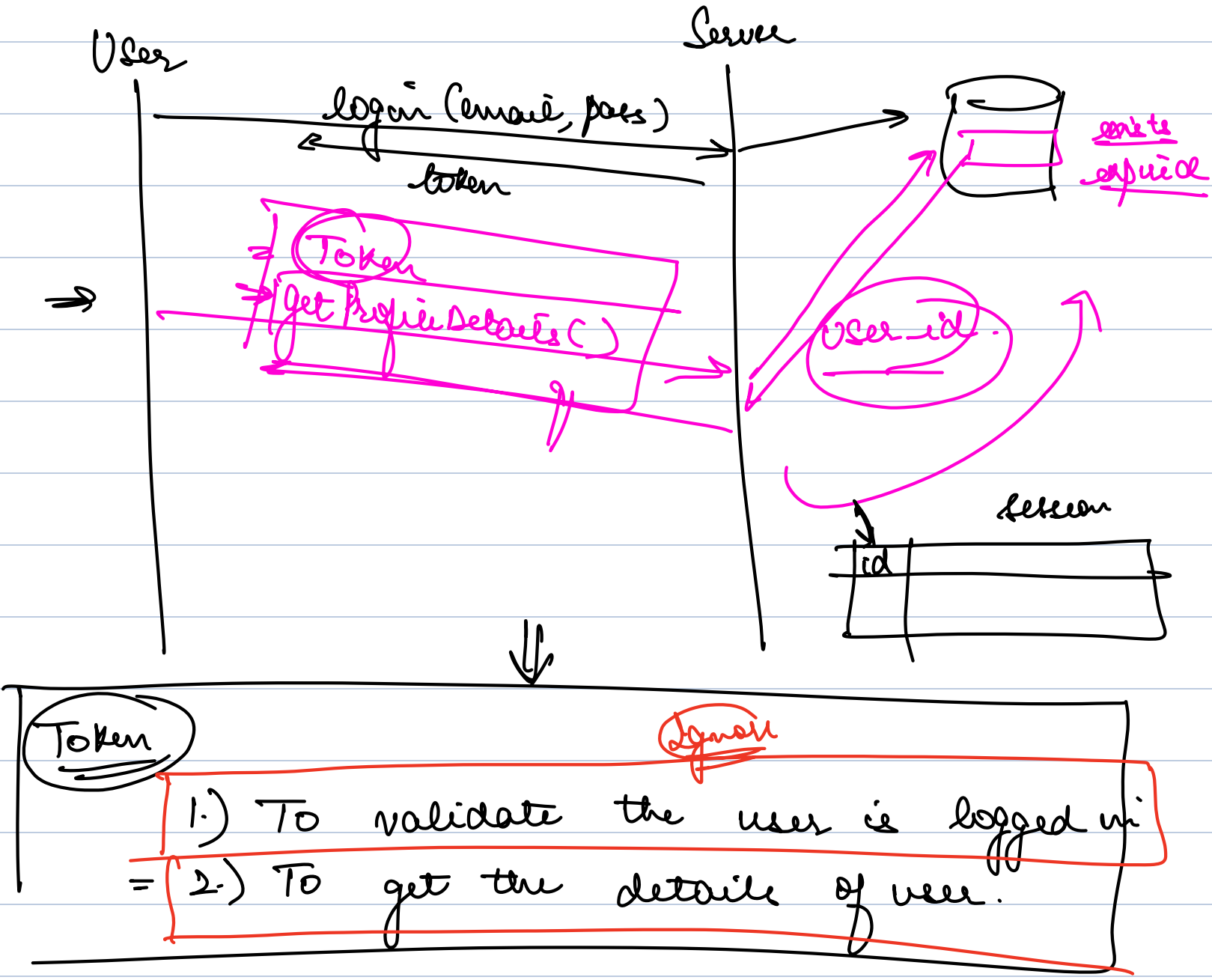




Better way to generate tokens !! Yes

JWT

JSON Web Tokens.

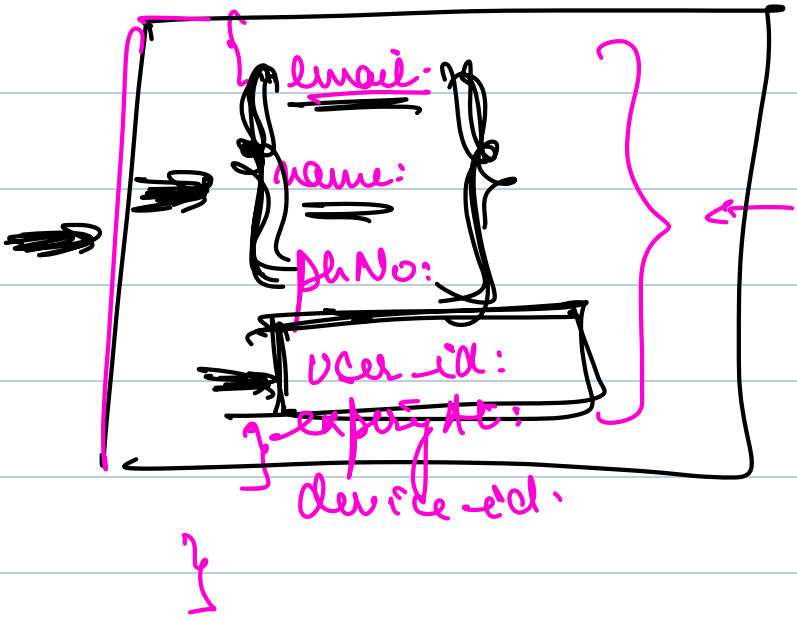


Can we do this in better way!

GET DETAILS OF USER

→ what if in token only user details are there?

a) create a json



put all attrs that you might want to fetch from token w/o needing to go to DB

}

loginC) {

j → createJSONC);

s = encodeJSONToString(j)

Serveeible

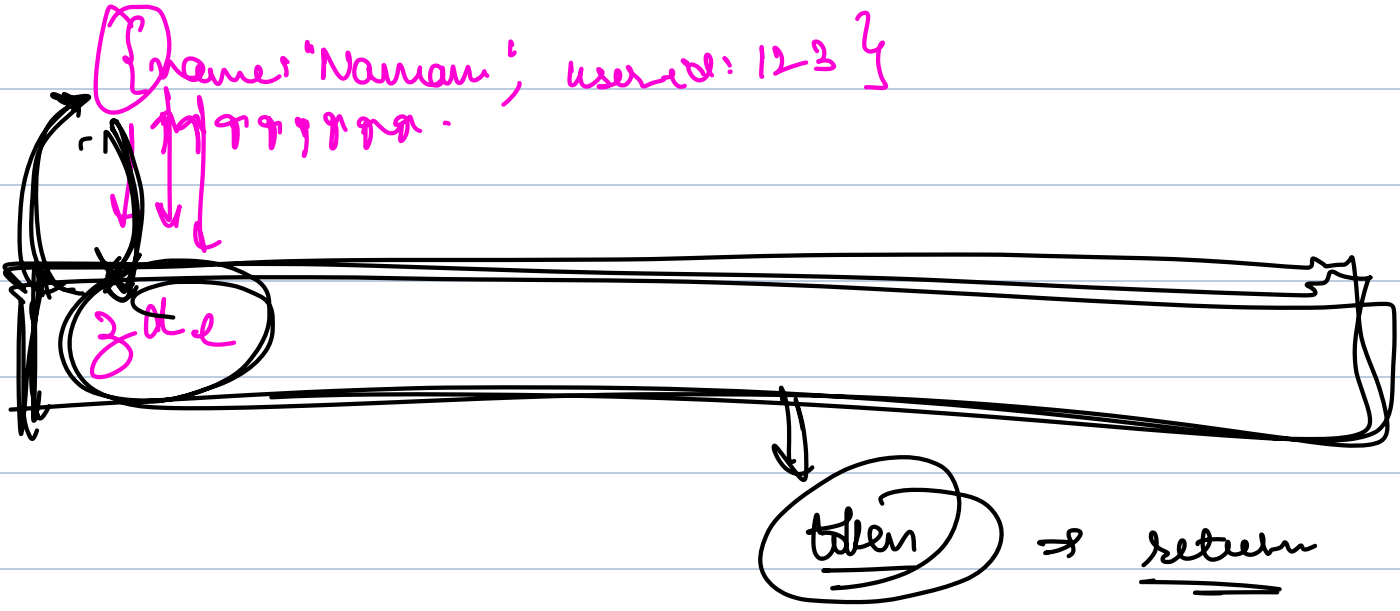
token = s

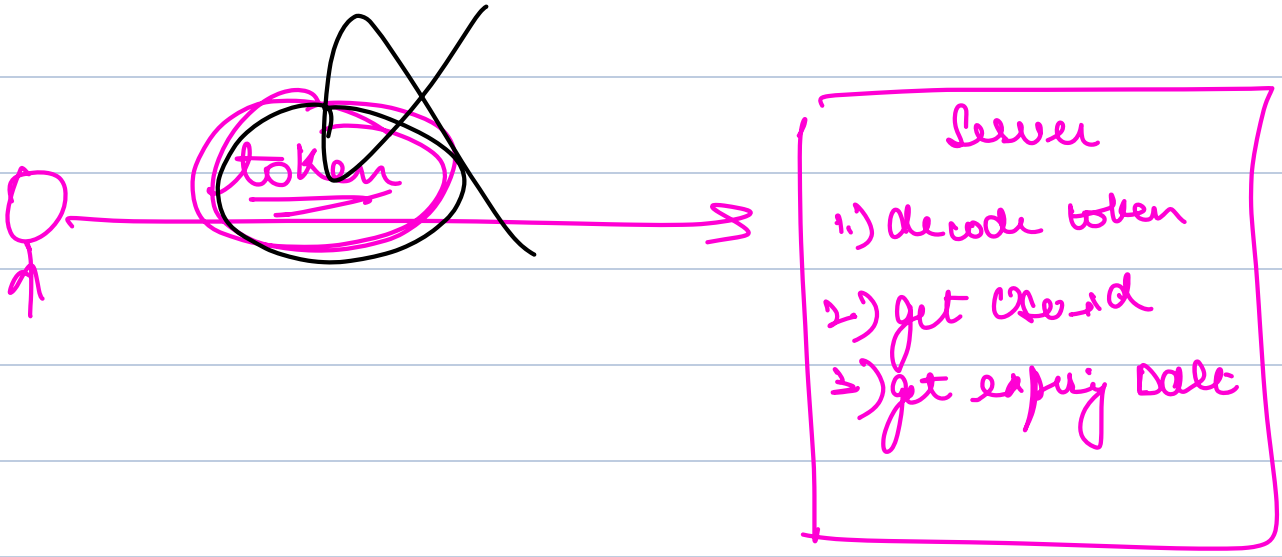
return token

}

e → name → string

Base 32

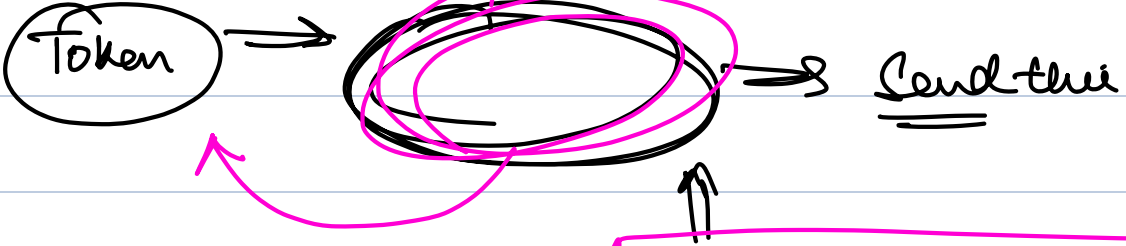




Token ✓

Safe ✗

Manipulate ✗



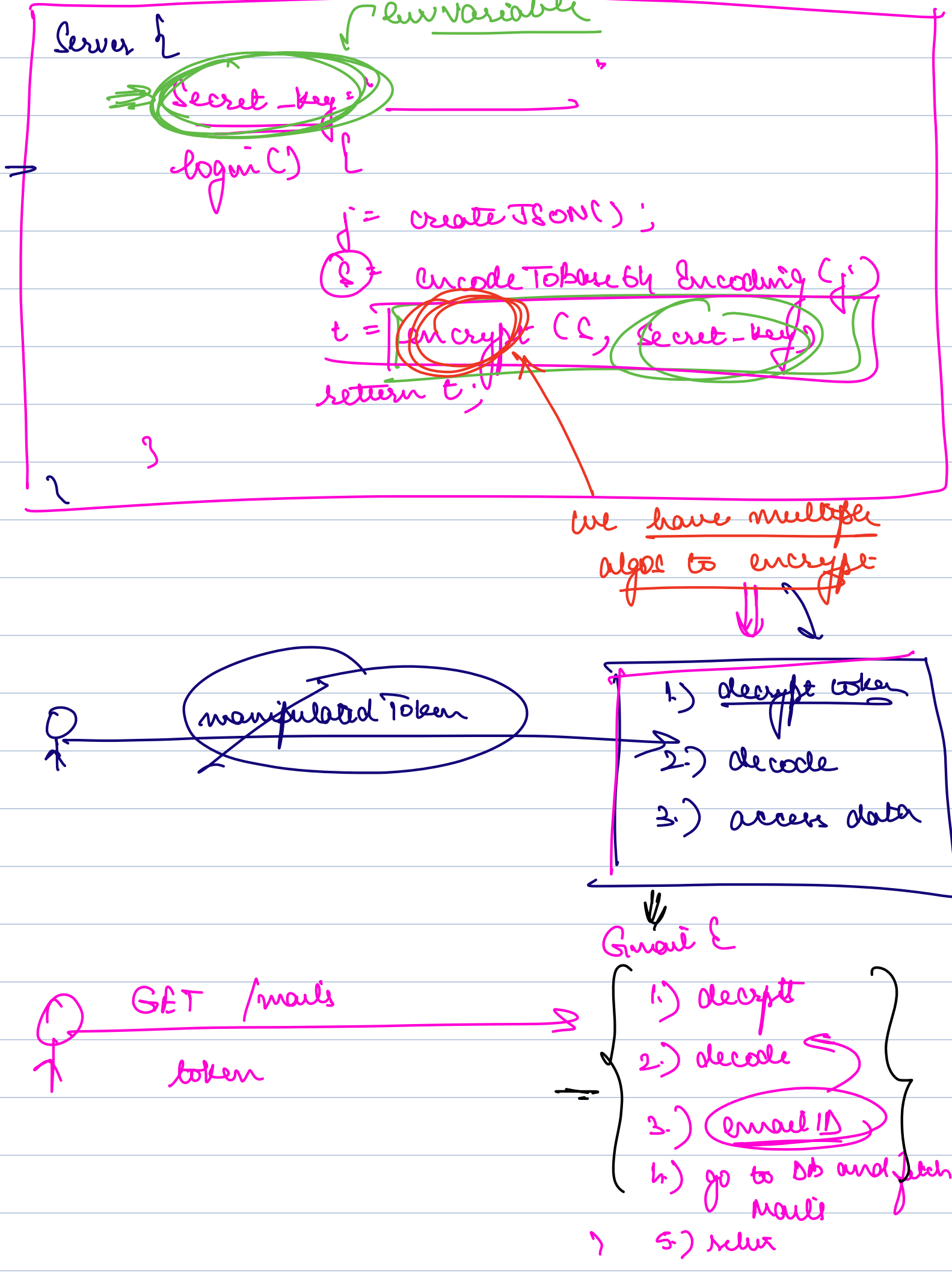
→ Something that a user can't do on their end

→ only a server can do at their end.

Encryption

ENCODING + PASSWORD → Secret key

→ you can decode BUT IFF you have the password.

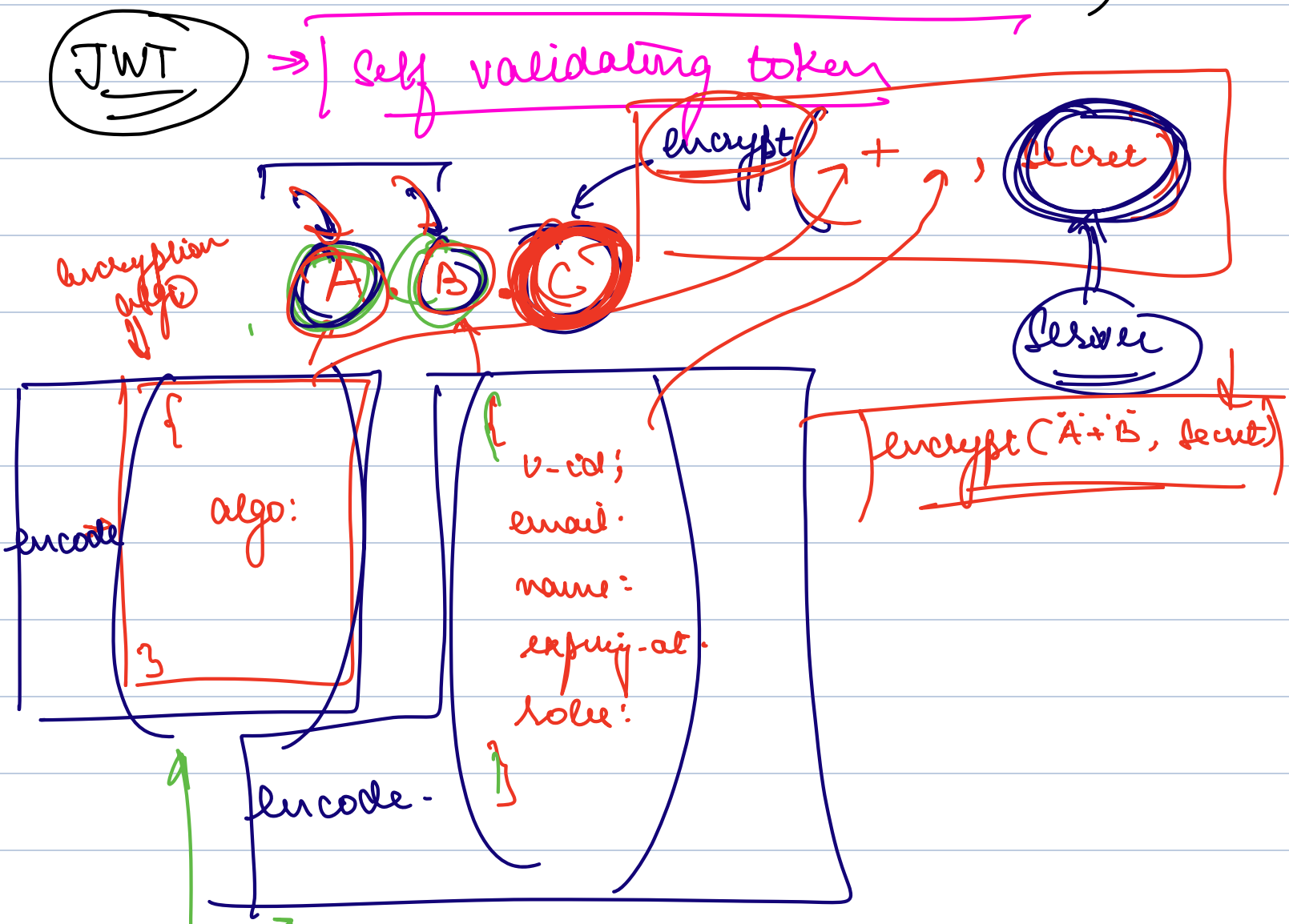

Server {
env variable
secret_key = "..."
login() {
j = createJSON();
encodeToBase64Encoding(j)
t = encrypt(t, secret_key)
return t;
}
}
we have multiple algos to encrypt
manipulated Token
1.) decrypt token
2.) decode
3.) access data
Gmail {
GET /mails
token
1.) decrypt
2.) decode
3.) email ID
4.) go to DB and fetch mails
5.) send

"Validating the token on server itself"

⇒ reduce latency

⇒ make req that require auth fast.

JWT ⇒ Self validating token



→ Only encoding

→ no encryption

→ user can decode at

⇒ even if user manipulate part A and B, they can't gen c

their end.

→ Don't put any confidential info here

as it requires Secret Key

login() {

j1 = create JSON of Algo();

S1 = encode(J1)

j2 = create JSON of User();

S2 = encode(J2)

S3 = encrypt(S1 + S2, secret)

return S1 + "." + S2 + "." + S3 ← Token

}

(S1.S2.S3)

→ fetch Emails (token) {

S4 = decrypt(S3, secret)

if (unable to decrypt) OR {

(S4 != S1 + S2)

Manipulation !! → reject

} else {

j2 = decode(S2)

u_id = j2.u_id

}

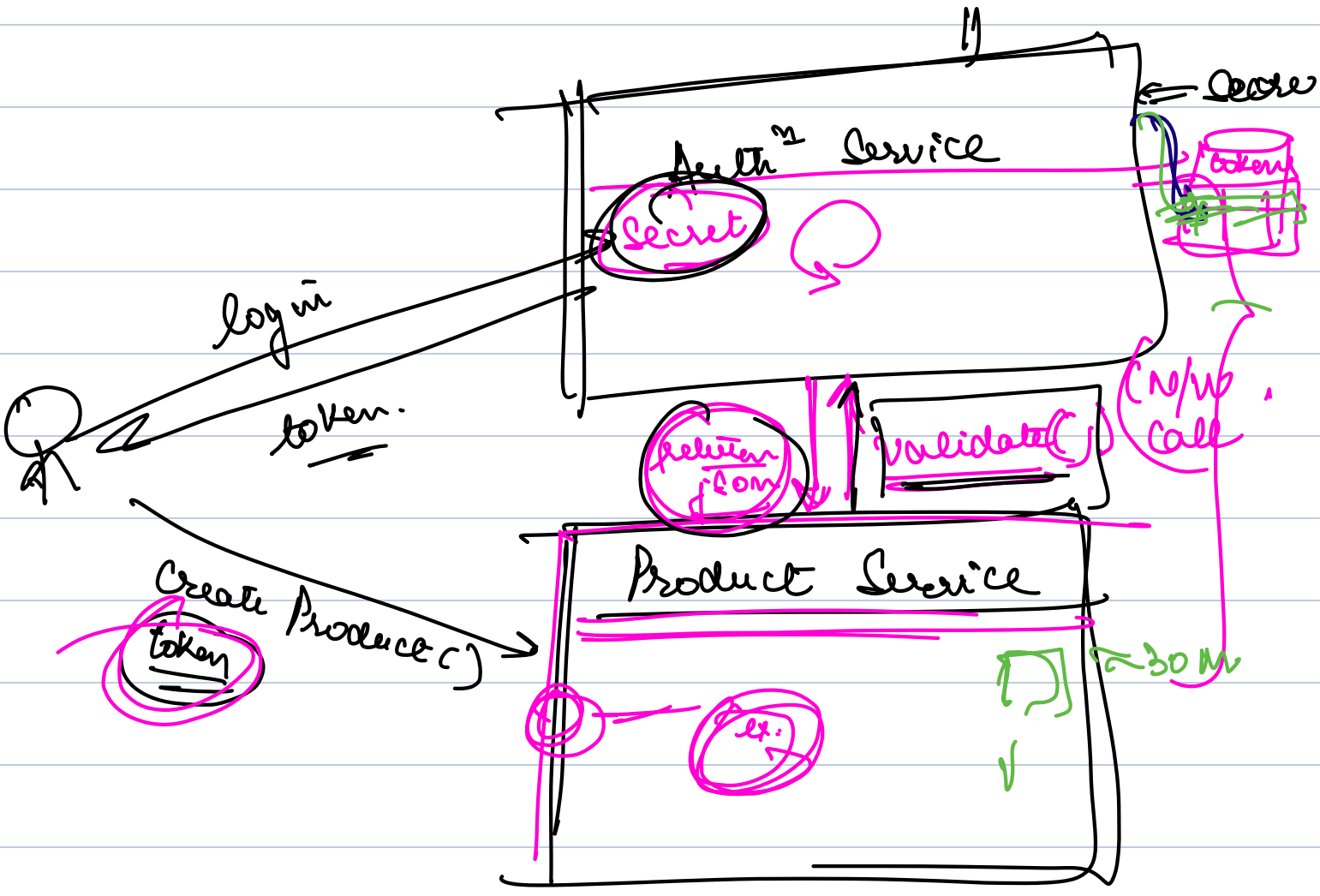
HLD

Monolithic ⇒ Microservices.

↓ Service By Service

⇒ pick up part that is most commonly accessed by other into an independent codebase.

99.9% ⇒ [auth module]



Features of Auth^n Service

1.) Session limit

2.) Logout from all device

≈ 30 min to Reflect

Latency

Openapi

OAuth

Auth^n Service

Validate

Google's Auth Serv.

Product Service

Payment Services

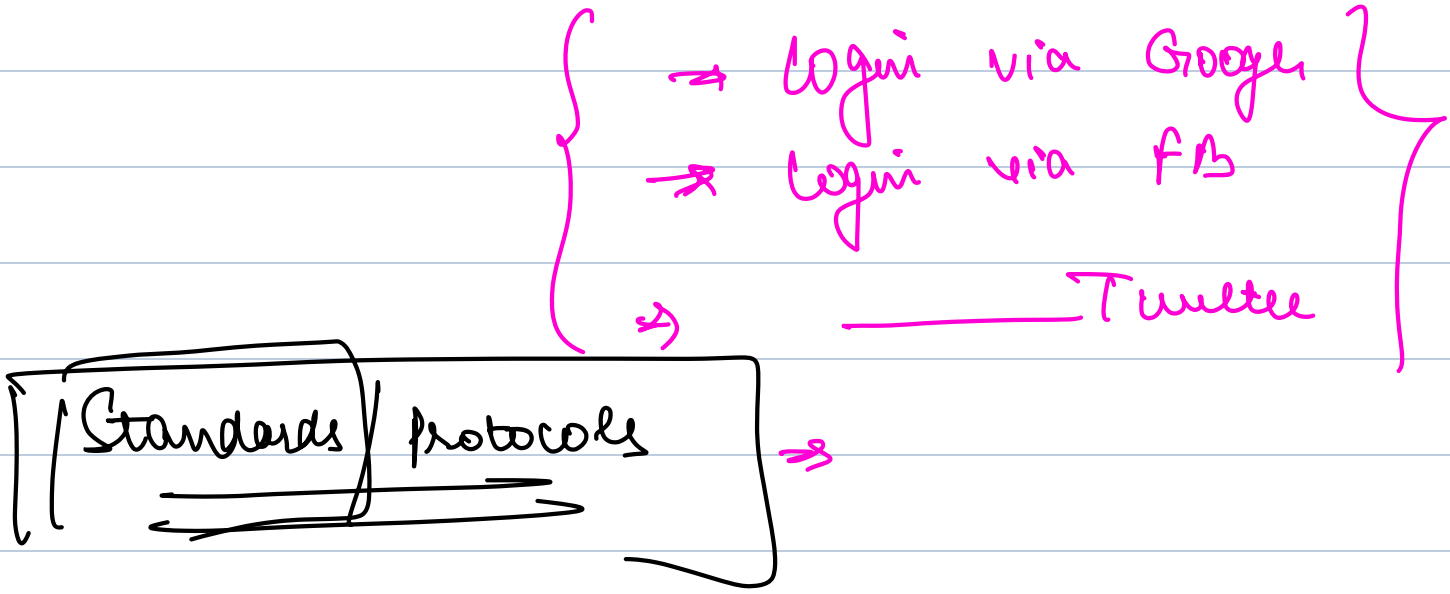
Scaler

→ Discourse

Community

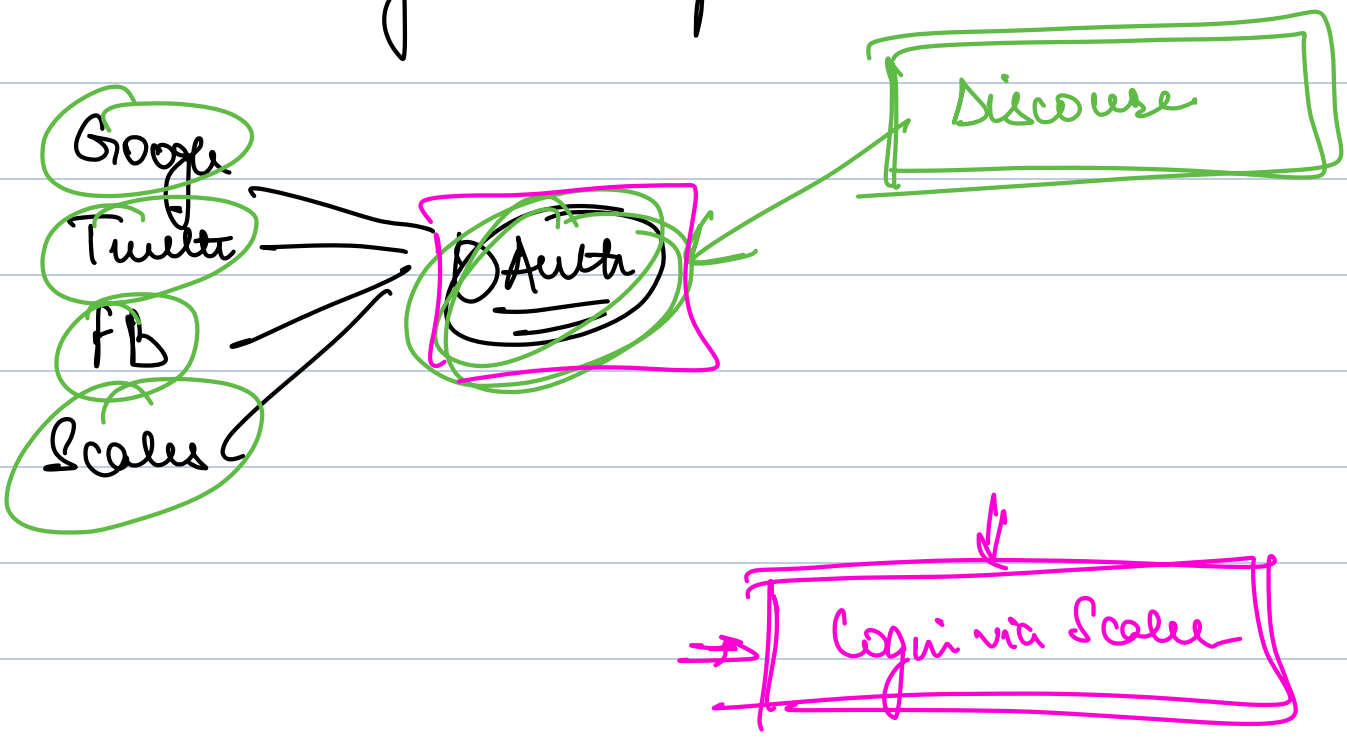
- Scaler
- Coma

Discourse



⇒ OAuth Standard ⇒ Managing authentication and authorization

⇒ Creates a standard API that can be implemented by any auth provides.



How OAuth Works

4 Participants

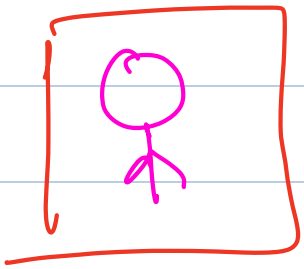
- User
- Application
- Resource Server
- Authorization Server

Resource Server → the owner of the info that we want to fetch

GMail Server

Authorization Server → Server that manages access to resource on resource ser

Google's Auth Server

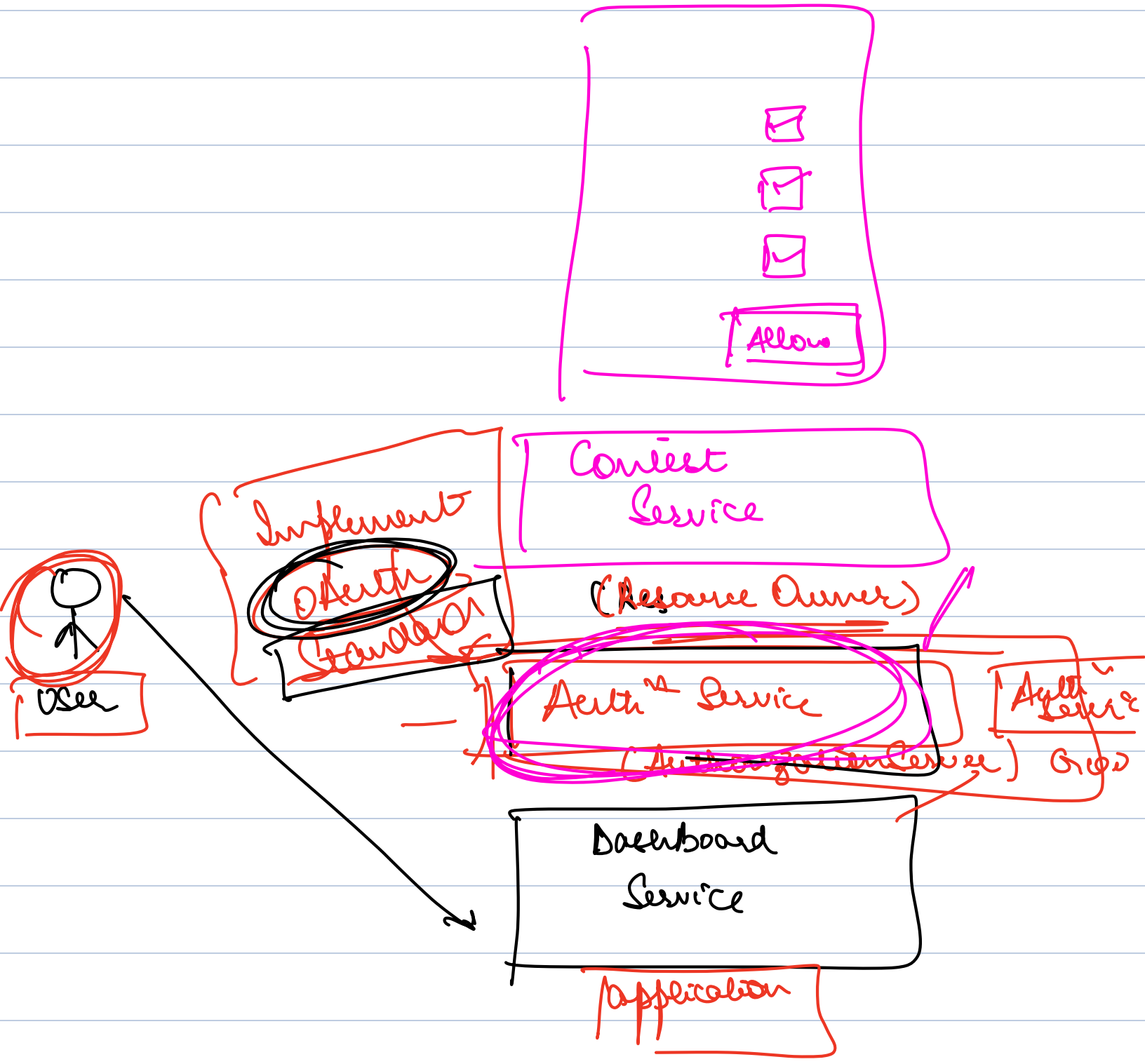


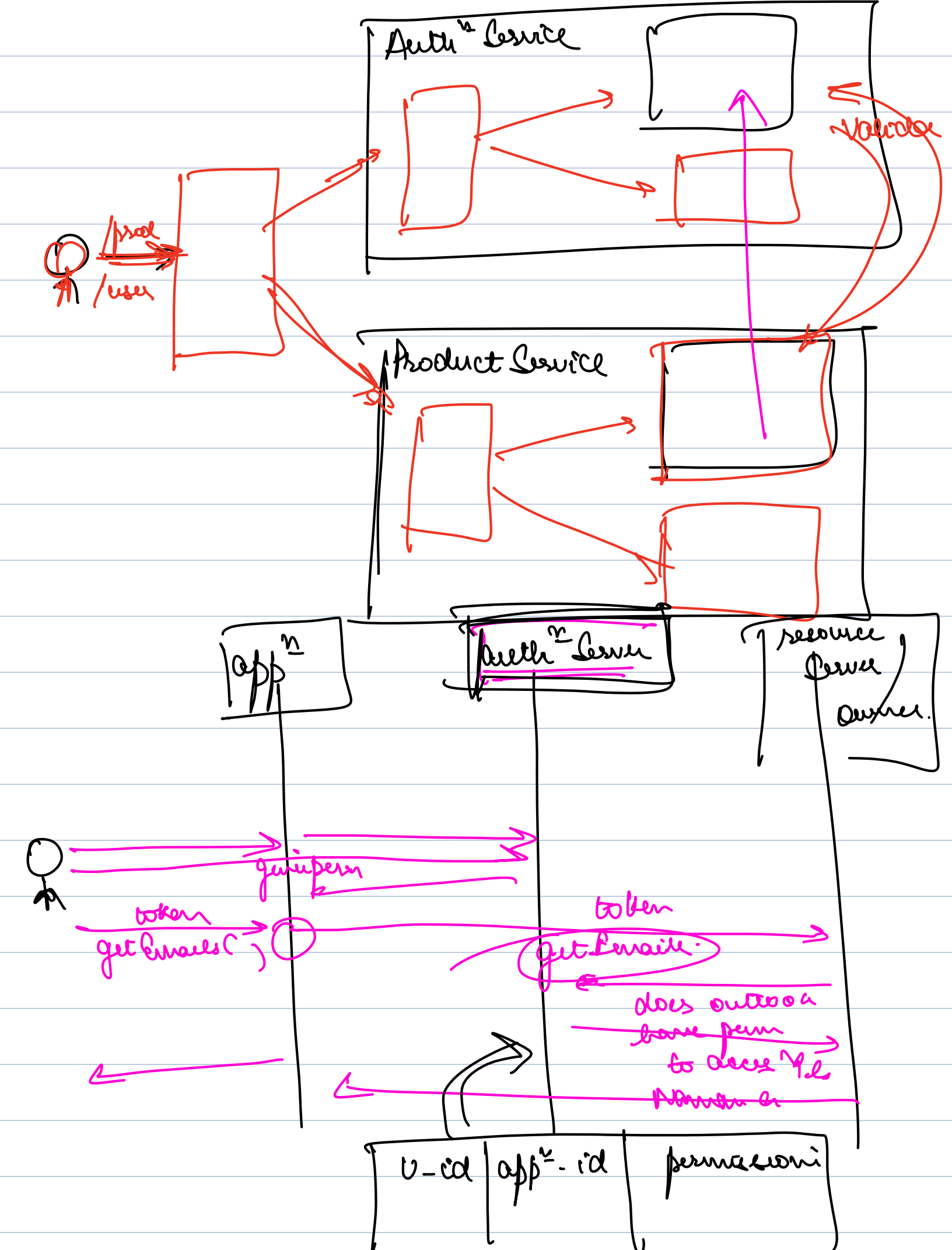
→ the person who wants to take an action

Application

Outlook

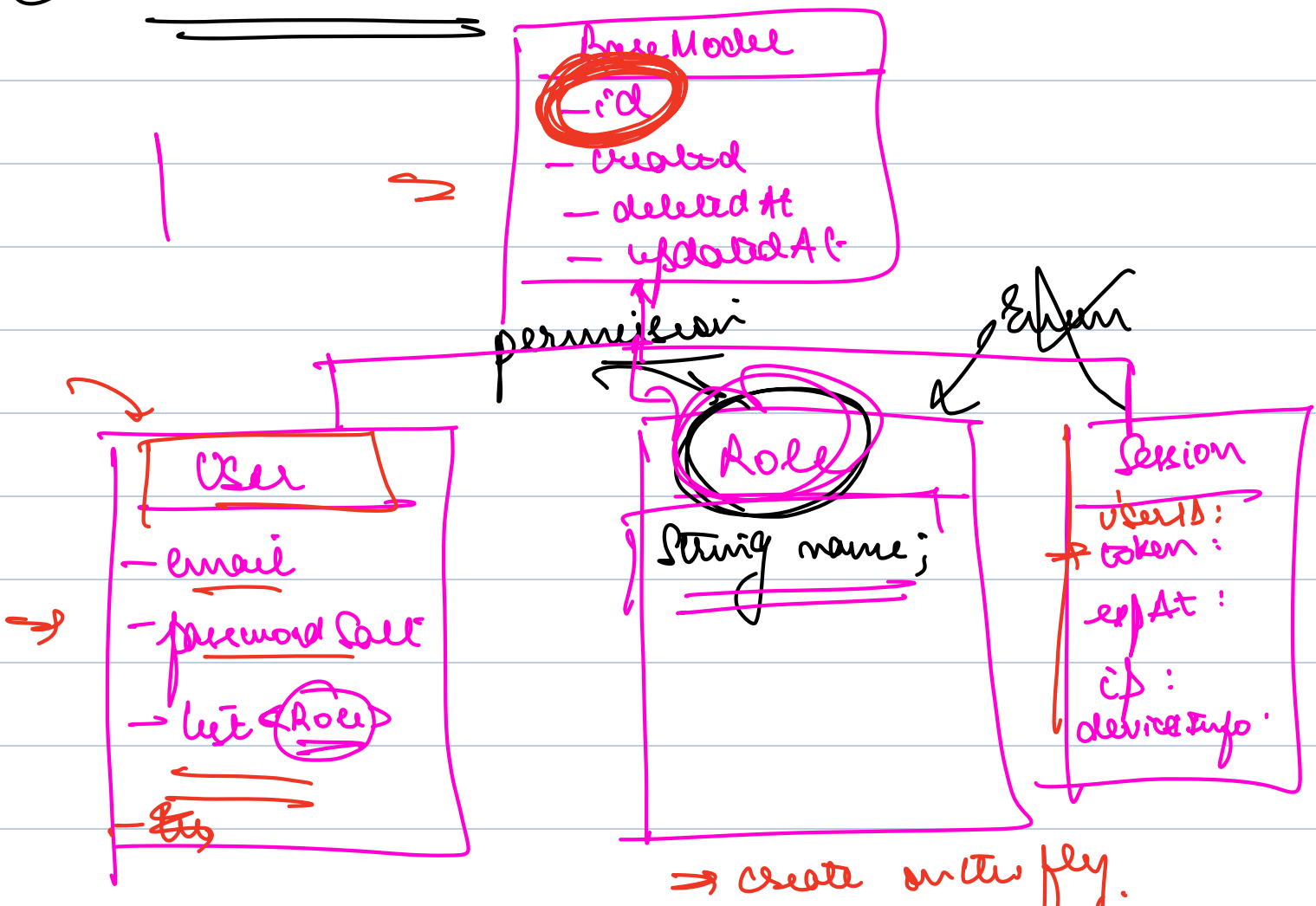
Example: (OUTLOOK) Mobile App that allows you to sign in via google and see your google mails.


Allow
Content Service
Implement
OAuth
Standard
(Resource Owner)
User
Auth^n Service
Dashboard Service
Application


Auth Service
Validate
prod
/user
Product Service
App
Auth Server
resource Server
owner.
getEmails()
token
getEmails()
token
getEmails
does outlook have perm to access Yash's emails
u-id
app-id
permissions

LLD of User Service

① Decide Models



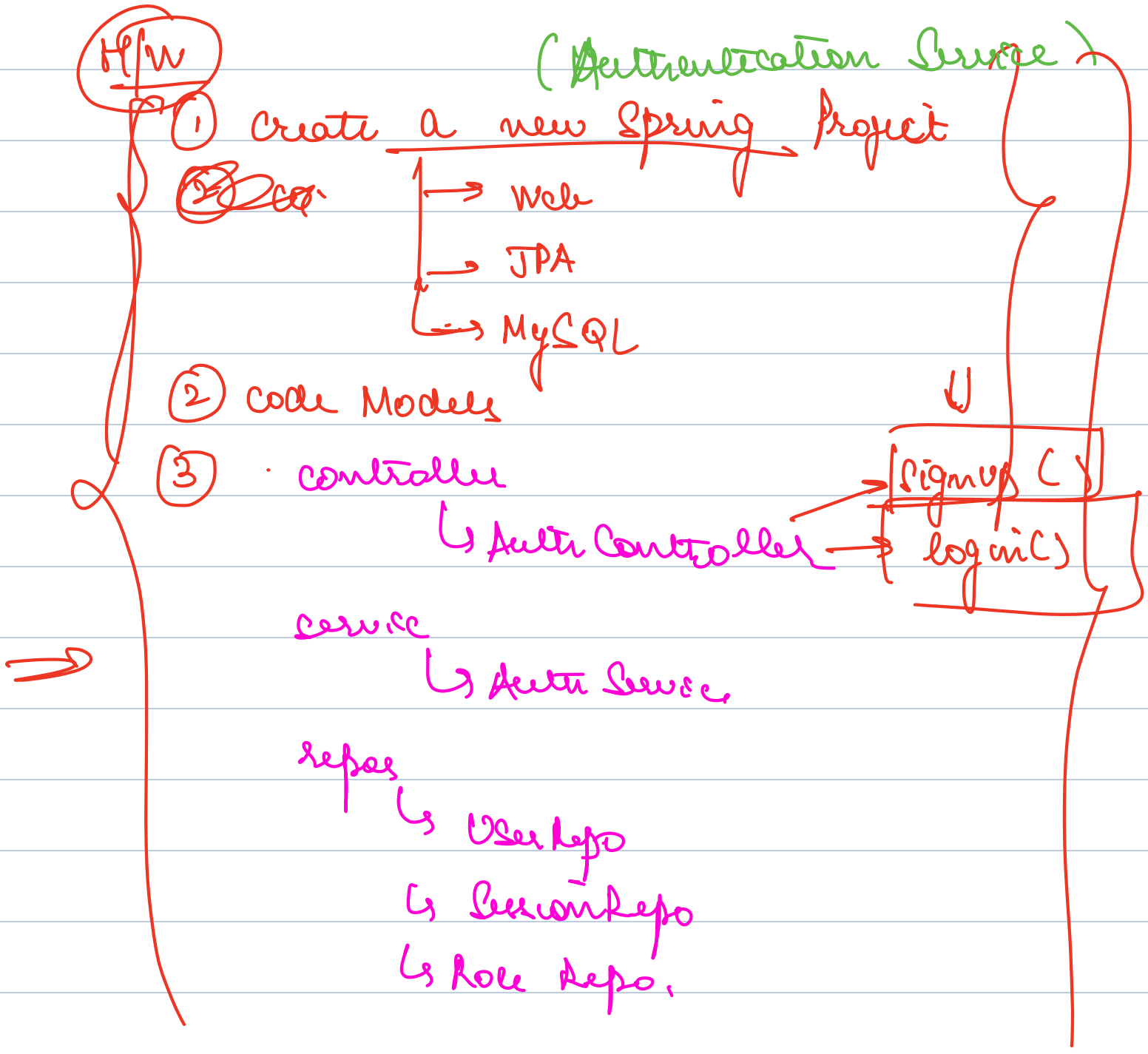
⇒ create enum fly.

{ PHOTOS_READ
- CREATE
MAIL_READ
MAIL_SEND }

① Signup

② updateRole(U_id, [roles])

(3) login


H/W
(Authentication Service)
1 Create a new Spring Project
Web
JPA
MySQL
2 Code Models
3 · controller
Auth Controller
signup()
login()
service
Auth Service
repos
User Repo
Session Repo
Role Repo.

1